
Windows User and Permission Management Lab

Course: Operating Systems and You: Becoming a Power User

Module: Users, Groups and Permissions

Date: 20/08/2026

Author: Florencia Horita

1. Introducción

Este laboratorio tiene como objetivo aplicar conceptos de administración de usuarios, grupos y permisos en Windows mediante PowerShell.

Se creó un escenario que simula la administración básica de recursos dentro de una organización. Para ello, se configuraron diferentes cuentas de usuario y grupos locales y posteriormente se asignaron permisos NTFS sobre una estructura de directorios.

Durante el laboratorio se practicó la creación y administración de usuarios y grupos locales, la asignación de membresías, la inspección de Access Control Lists (ACL), la configuración de permisos mediante `icacls` y el funcionamiento de la herencia de permisos entre directorios.

El objetivo principal fue comprender la relación entre usuarios, grupos, recursos y permisos, utilizando un modelo de administración similar al empleado en entornos Windows reales.

2. Entorno Utilizado

El laboratorio fue realizado utilizando el siguiente entorno:

- Windows 11 Pro virtualizado
- PowerShell
- Sistema de archivos NTFS
- Host macOS
- Máquina virtual Windows

Las operaciones administrativas fueron realizadas desde PowerShell con los privilegios necesarios para crear cuentas locales, grupos y modificar permisos sobre el sistema de archivos.

3. Escenario del Laboratorio

Para simular un entorno de trabajo se definieron tres usuarios locales:

- **Ana** — usuario perteneciente al grupo general del laboratorio.
- **Carlos** — usuario perteneciente al equipo de proyectos.
- **AdminLab** — usuario perteneciente al grupo administrativo del laboratorio.

También se definieron tres grupos locales:

- **LabUsers**

- `ProjectTeam`
- `LabAdmins`

La relación entre usuarios y grupos utilizada durante el laboratorio fue:

`Ana` → `LabUsers`

`Carlos` → `ProjectTeam`

`AdminLab` → `LabAdmins`

Posteriormente, estos grupos fueron utilizados para administrar el acceso a diferentes recursos del sistema de archivos.

El uso de grupos permite asignar permisos según funciones o roles, evitando la necesidad de configurar individualmente cada usuario.

4. Creación de Usuarios Locales

Se crearon las cuentas locales desde PowerShell utilizando `New-LocalUser`.

Para introducir las contraseñas sin escribirlas directamente como texto visible en los comandos se utilizó `Read-Host` junto con el parámetro `-AsSecureString`.

Comandos utilizados

```
$password = Read-Host "Enter password for Ana"
-AsSecureString
New-LocalUser -Name "Ana" -Password $password
```

Posteriormente se verificó la creación de la cuenta mediante:

```
Get-LocalUser -Name "Ana"
```

El resultado mostró `Enabled = True`, indicando que la cuenta se encontraba habilitada.

El mismo procedimiento fue utilizado posteriormente para crear las cuentas `Carlos` y `AdminLab`.

Conceptos aplicados

- Creación de usuarios locales
- Manejo de contraseñas mediante `SecureString`
- Verificación de cuentas locales
- Estado habilitado o deshabilitado de una cuenta
- Diferencia entre una cuenta habilitada y los privilegios administrativos

Figura 1 — Creación y verificación del usuario local Ana

```
PS C:\WINDOWS\system32> $password = Read-Host "Enter password for Ana" -AsSecureString
Enter password for Ana: ***
PS C:\WINDOWS\system32> New-LocalUser -Name "Ana" -Password $password

Name Enabled Description
----
Ana    True

PS C:\WINDOWS\system32>
```

5. Creación de Grupos Locales

Para organizar los usuarios según sus funciones dentro del escenario se crearon tres grupos locales.

LabUsers

Primero se creó **LabUsers**, destinado a representar a los usuarios estándar del laboratorio.

Comandos utilizados

```
New-LocalGroup -Name "LabUsers" -Description "Standard users for the lab"
```

```
Get-LocalGroup -Name "LabUsers"
```

El segundo comando permitió comprobar que el grupo había sido creado correctamente.

Figura 2 — Creación y verificación del grupo local LabUsers

```
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> New-LocalGroup -Name "LabUsers" -Description "Standard users for the lab"

Name      Description
----      -
LabUsers  Standard users for the lab

PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Get-LocalGroup -Name "LabUsers"

Name      Description
----      -
LabUsers  Standard users for the lab
```

ProjectTeam

Posteriormente se creó **ProjectTeam**, destinado a los usuarios responsables de los recursos relacionados con proyectos.

```
New-LocalGroup -Name "ProjectTeam" -Description "Users
responsible for project resources"
Get-LocalGroup -Name "ProjectTeam"
```

Figura 3 — Creación y verificación del grupo local ProjectTeam

```
PS C:\WINDOWS\system32> New-LocalGroup -Name "ProjectTeam" -Description "Users responsible for project resources"

Name      Description
----      -
ProjectTeam Users responsible for project resources

PS C:\WINDOWS\system32> Get-LocalGroup -Name "ProjectTeam"

Name      Description
----      -
ProjectTeam Users responsible for project resources
```

LabAdmins

Finalmente se creó LabAdmins, utilizado como grupo administrativo dentro del escenario del laboratorio.

```
New-LocalGroup -Name "LabAdmins" -Description "IT
administrators for the lab"
Get-LocalGroup -Name "LabAdmins"
```

Figura 4 — Creación y verificación del grupo local LabAdmins

```
PS C:\WINDOWS\system32> New-LocalGroup -Name "LabAdmins" -Description "IT administrators for the lab"

Name      Description
----      -
LabAdmins IT administrators for the lab

PS C:\WINDOWS\system32> Get-Localgroup -Name "LabAdmins"

Name      Description
----      -
LabAdmins IT administrators for the lab
```

Conceptos aplicados

- Creación de grupos locales
- Organización de usuarios según funciones
- Uso de descripciones para identificar el propósito de los grupos
- Verificación de grupos mediante Get-LocalGroup

6. Administración de Membresías

Una vez creados los usuarios y los grupos, cada usuario fue incorporado al grupo correspondiente.

Ana → LabUsers

Ana fue agregada al grupo LabUsers.

```
Add-LocalGroupMember -Group "LabUsers" -Member "Ana"  
Get-LocalGroupMember -Group "LabUsers"
```

El segundo comando permitió verificar que Ana aparecía como miembro del grupo.

Figura 5 — Incorporación de Ana al grupo LabUsers

```
PS C:\WINDOWS\system32> Add-LocalGroupMember -Group "LabUsers" -Member "Ana"  
PS C:\WINDOWS\system32> Get-LocalGroupMember -Group "LabUsers"
```

ObjectClass	Name	PrincipalSource
User	WIN-SFMECCHAUKE\Ana	Local

Carlos → ProjectTeam

Posteriormente, Carlos fue agregado al grupo ProjectTeam.

```
Add-LocalGroupMember -Group "ProjectTeam" -Member "Carlos"  
Get-LocalGroupMember -Group "ProjectTeam"
```

Figura 6 — Incorporación de Carlos al grupo ProjectTeam

```
PS C:\WINDOWS\system32> Add-LocalGroupMember -Group "ProjectTeam" -Member "Carlos"  
PS C:\WINDOWS\system32> Get-LocalGroupMember -Group "ProjectTeam"
```

ObjectClass	Name	PrincipalSource
User	WIN-SFMECCHAUKE\Carlos	Local

AdminLab → LabAdmins

Finalmente, AdminLab fue agregado al grupo LabAdmins.

```
Add-LocalGroupMember -Group "LabAdmins" -Member "AdminLab"  
Get-LocalGroupMember -Group "LabAdmins"
```

Figura 7 — Incorporación de AdminLab al grupo LabAdmins

```
PS C:\WINDOWS\system32> Add-LocalGroupMember -Group "LabAdmins" -Member "AdminLab"
PS C:\WINDOWS\system32> Get-LocalGroupMember -Group "LabAdmins"

ObjectClass Name PrincipalSource
-----
User WIN-SFMECCHAUKE\AdminLab Local
```

Es importante señalar que pertenecer al grupo `LabAdmins` no convierte automáticamente a `AdminLab` en administrador de Windows. `LabAdmins` es un grupo creado específicamente para este laboratorio y es diferente del grupo integrado `BUILTIN\Administrators`.

7. Verificación General de Membresías

Después de realizar las asignaciones se verificó conjuntamente la configuración de los tres grupos.

Comandos utilizados

```
Get-LocalGroupMember -Group "LabUsers"
Get-LocalGroupMember -Group "ProjectTeam"
Get-LocalGroupMember -Group "LabAdmins"
```

La configuración obtenida fue:

Ana → LabUsers
Carlos → ProjectTeam
AdminLab → LabAdmins

Esta verificación permitió confirmar que la estructura de usuarios y grupos había quedado preparada correctamente antes de comenzar la administración de permisos.

Conceptos aplicados

- Membresía de grupos
- Administración de usuarios mediante grupos
- Verificación de membresías
- Separación entre usuarios, grupos y permisos

Figura 8 — Verificación general de las membresías de los grupos

```
PS C:\WINDOWS\system32> Get-LocalGroupMember -Group "LabUsers"

ObjectClass Name                PrincipalSource
-----
User          WIN-SFMECCHAUKEK\Ana Local

PS C:\WINDOWS\system32> Get-LocalGroupMember -Group "ProjectTeam"

ObjectClass Name                PrincipalSource
-----
User          WIN-SFMECCHAUKEK\Carlos Local

PS C:\WINDOWS\system32> Get-LocalGroupMember -Group "LabAdmins"

ObjectClass Name                PrincipalSource
-----
User          WIN-SFMECCHAUKEK\AdminLab Local
```

8. Creación de la Estructura de Recursos

Para aplicar y analizar permisos NTFS se creó una estructura de directorios que representa diferentes recursos de una organización.

La estructura utilizada fue:

```
C:\CompanyLab
├── Public
├── Documents
├── Projects
│   ├── ProjectA
│   └── ProjectB
```

Los directorios fueron creados mediante `New-Item`.

Comandos utilizados

```
New-Item -Path "C:\CompanyLab" -ItemType Directory
New-Item -Path "C:\CompanyLab\Public" -ItemType Directory
New-Item -Path "C:\CompanyLab\Documents" -ItemType Directory
New-Item -Path "C:\CompanyLab\Projects" -ItemType Directory
New-Item -Path "C:\CompanyLab\Projects\ProjectA" -ItemType Directory
New-Item -Path "C:\CompanyLab\Projects\ProjectB" -ItemType Directory
```

Posteriormente se verificó la estructura completa mediante:

```
Get-ChildItem "C:\CompanyLab" -Recurse
```

El parámetro `-Recurse` permitió visualizar también los directorios ubicados dentro de `Projects`.

Conceptos aplicados

- Creación de directorios
- Organización jerárquica de recursos
- Uso de rutas
- Verificación recursiva de una estructura de directorios

Figura 9 — Verificación de la estructura de directorios de CompanyLab

```
PS C:\WINDOWS\system32> Get-ChildItem "C:\CompanyLab" -Recurse

Directory: C:\CompanyLab

Mode                LastWriteTime         Length Name
----                -
d-----            8/11/2026 12:13 PM             Documents
d-----            8/11/2026 12:15 PM             Projects
d-----            8/11/2026 12:11 PM             Public

Directory: C:\CompanyLab\Projects

Mode                LastWriteTime         Length Name
----                -
d-----            8/11/2026 12:15 PM             ProjectA
d-----            8/11/2026 12:15 PM             ProjectB
```

9. Inspección de la ACL Predeterminada

Antes de realizar modificaciones sobre los permisos se inspeccionó la Access Control List de `CompanyLab`.

Inicialmente se utilizó:

```
Get-Acl "C:\CompanyLab"
```

La salida permitió observar información general como el propietario de la carpeta y sus entradas de acceso.

Para visualizar individualmente las entradas de la ACL se utilizó:

```
(Get-Acl "C:\CompanyLab").Access
```

La inspección mostró diferentes identidades configuradas por Windows, entre ellas:

- BUILTIN\Administrators
- NT AUTHORITY\SYSTEM
- BUILTIN\Users
- NT AUTHORITY\Authenticated Users

También se analizaron propiedades como:

- FileSystemRights
- AccessControlType
- IdentityReference
- IsInherited
- InheritanceFlags
- PropagationFlags

Por ejemplo, BUILTIN\Administrators disponía de FullControl, mientras que BUILTIN\Users presentaba permisos más limitados como ReadAndExecute.

La propiedad IsInherited = True permitió identificar que determinadas entradas habían sido heredadas desde un directorio superior.

También se observaron indicadores relacionados con la propagación de permisos:

- ObjectInherit — herencia hacia archivos.
- ContainerInherit — herencia hacia subdirectorios.
- InheritOnly — entrada destinada a ser heredada por objetos descendientes.

Conceptos aplicados

- ACL (Access Control List)
- DACL (Discretionary Access Control List)
- Entradas de control de acceso
- Allow
- FullControl
- ReadAndExecute
- Permisos heredados
- ObjectInherit
- ContainerInherit
- InheritOnly

Figura 10 — Inspección de las entradas de acceso de la ACL de CompanyLab

```
PS C:\WINDOWS\system32> (Get-Acl "C:\CompanyLab").Access

FileSystemRights : FullControl
AccessControlType : Allow
IdentityReference : BUILTIN\Administrators
IsInherited : True
InheritanceFlags : ContainerInherit, ObjectInherit
PropagationFlags : None

FileSystemRights : FullControl
AccessControlType : Allow
IdentityReference : NT AUTHORITY\SYSTEM
IsInherited : True
InheritanceFlags : ContainerInherit, ObjectInherit
PropagationFlags : None

FileSystemRights : ReadAndExecute, Synchronize
AccessControlType : Allow
IdentityReference : BUILTIN\Users
IsInherited : True
InheritanceFlags : ContainerInherit, ObjectInherit
PropagationFlags : None

FileSystemRights : Modify, Synchronize
AccessControlType : Allow
IdentityReference : NT AUTHORITY\Authenticated Users
IsInherited : True
InheritanceFlags : None
PropagationFlags : None

FileSystemRights : -536805376
AccessControlType : Allow
IdentityReference : NT AUTHORITY\Authenticated Users
IsInherited : True
InheritanceFlags : ContainerInherit, ObjectInherit
PropagationFlags : InheritOnly
```

10. Configuración de Permisos para LabUsers

Una vez inspeccionados los permisos existentes, se comenzó a configurar el acceso para los grupos creados durante el laboratorio.

Al grupo `LabUsers` se le otorgó permiso de lectura y ejecución sobre el directorio `Public`.

Comando utilizado

```
icaccls "C:\CompanyLab\Public" /grant "LabUsers:(OI)(CI)RX"
```

En esta configuración:

- /grant permite otorgar un permiso.
- LabUsers identifica al grupo que recibe el permiso.
- RX representa Read & Execute.
- OI representa Object Inherit.
- CI representa Container Inherit.

Por lo tanto, LabUsers recibió permiso de lectura y ejecución sobre Public, con capacidad de propagación hacia archivos y subdirectorios.

La configuración fue verificada mediante:

```
icaccls "C:\CompanyLab\Public"
```

Posteriormente también se inspeccionó la ACL mediante:

```
(Get-Acl "C:\CompanyLab\Public").Access
```

Esto permitió confirmar la presencia de LabUsers con el permiso correspondiente.

Conceptos aplicados

- Asignación de permisos NTFS
- Read & Execute
- Uso de /grant
- Object Inherit
- Container Inherit
- Verificación de permisos

Figura 11 — Asignación y verificación de Read & Execute para LabUsers sobre Public

```
PS C:\WINDOWS\system32> icaccls "C:\CompanyLab\Public" /grant "LabUsers:(OI)(CI)RX"
processed file: C:\CompanyLab\Public
Successfully processed 1 files; Failed processing 0 files
PS C:\WINDOWS\system32> icaccls "C:\CompanyLab\Public"
C:\CompanyLab\Public WIN-SFMECCHAUKEK\LabUsers:(OI)(CI)(RX)
                     BUILTIN\Administrators:(I)(OI)(CI)(F)
                     NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
                     BUILTIN\Users:(I)(OI)(CI)(RX)
                     NT AUTHORITY\Authenticated Users:(I)(M)
                     NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)
```

11. Configuración de Permisos para ProjectTeam

Al grupo `ProjectTeam` se le otorgó permiso `Modify` sobre el directorio `Projects`.

Comando utilizado

```
icaccls "C:\CompanyLab\Projects" /grant "ProjectTeam:(OI)(CI)M"
```

En esta configuración:

- `M` representa `Modify`.
- `OI` permite la herencia hacia archivos.
- `CI` permite la herencia hacia subdirectorios.

El permiso `Modify` proporciona un nivel de acceso superior a `Read & Execute`, permitiendo realizar modificaciones sobre el contenido según las reglas de acceso configuradas.

La configuración fue verificada mediante:

```
icaccls "C:\CompanyLab\Projects"
```

La salida permitió confirmar que `ProjectTeam` disponía del permiso configurado sobre el directorio.

Conceptos aplicados

- Permiso `Modify`
- Asignación de permisos a grupos
- Herencia hacia archivos
- Herencia hacia subdirectorios
- Verificación mediante `icaccls`

Figura 12 — Asignación y verificación de Modify para ProjectTeam sobre Projects

```
PS C:\WINDOWS\system32> icaccls "C:\CompanyLab\Projects" /grant "ProjectTeam:(OI)(CI)M"
processed file: C:\CompanyLab\Projects
Successfully processed 1 files; Failed processing 0 files
PS C:\WINDOWS\system32> icaccls "C:\CompanyLab\Projects"
C:\CompanyLab\Projects  WIN-SFMECCHAUKE\ProjectTeam:(OI)(CI)(M)
                        BUILTIN\Administrators:(I)(OI)(CI)(F)
                        NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
                        BUILTIN\Users:(I)(OI)(CI)(RX)
                        NT AUTHORITY\Authenticated Users:(I)(M)
                        NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)
```

12. Configuración de Permisos para LabAdmins

Para representar un grupo con mayor nivel de acceso sobre los recursos del laboratorio, se otorgó **Full Control** a **LabAdmins** sobre **CompanyLab**.

Comando utilizado

```
icacls "C:\CompanyLab" /grant "LabAdmins:(OI)(CI)F"
```

En esta configuración:

- **F** representa **Full Control**.
- **OI** permite la herencia hacia archivos.
- **CI** permite la herencia hacia subdirectorios.

Posteriormente se verificó la configuración mediante:

```
icacls "C:\CompanyLab"
```

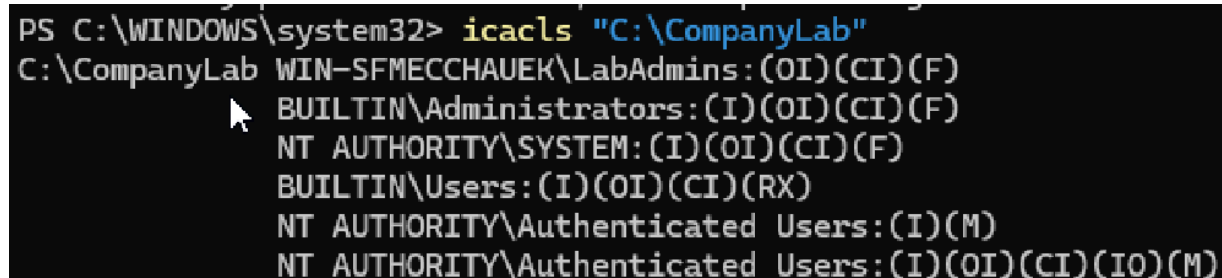
Esto permitió comprobar que **LabAdmins** había recibido **Full Control** sobre el recurso.

El grupo **LabAdmins** debe distinguirse de **BUILTIN\Administrators**. **LabAdmins** fue creado específicamente durante este laboratorio y sus capacidades dependen de los permisos que se le asignen.

Conceptos aplicados

- **Full Control**
- Administración de permisos mediante grupos
- Herencia de permisos
- Diferencia entre un grupo creado por el administrador y los grupos integrados de Windows

Figura 13 — Asignación y verificación de Full Control para LabAdmins sobre CompanyLab



```
PS C:\WINDOWS\system32> icacls "C:\CompanyLab"  
C:\CompanyLab WIN-SFMECCHAUER\LabAdmins:(OI)(CI)(F)  
                BUILTIN\Administrators:(I)(OI)(CI)(F)  
                NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)  
                BUILTIN\Users:(I)(OI)(CI)(RX)  
                NT AUTHORITY\Authenticated Users:(I)(M)  
                NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)
```

13. Verificación de la Herencia de Permisos

Después de otorgar **Modify** a **ProjectTeam** sobre **Projects**, se comprobó si el permiso se había propagado hacia uno de sus subdirectorios.

La estructura relevante era:

```
Projects
├── ProjectA
└── ProjectB
```

El permiso había sido configurado sobre **Projects** utilizando:

```
(OI)(CI)M
```

No se asignó directamente un permiso a **ProjectA**.

Para comprobar su ACL se utilizó:

```
icacls "C:\CompanyLab\Projects\ProjectA"
```

La entrada correspondiente a **ProjectTeam** mostró el indicador:

```
(I)
```

(I) representa **Inherited**, indicando que el permiso no había sido configurado directamente sobre **ProjectA**, sino que había sido heredado desde el directorio padre **Projects**.

La relación observada fue:

```
ProjectTeam
  |
  | Modify
  ↓
Projects
  |
  | herencia
  ↓
ProjectA
  └── Modify (Inherited)
```

Como **Carlos** pertenece a **ProjectTeam**, el modelo de acceso utilizado durante el laboratorio puede representarse como:

```
Carlos
  ↓
ProjectTeam
  ↓
Modify
```

↓
Projects
↓
ProjectA

Conceptos aplicados

- Herencia de permisos NTFS
- Permisos heredados
- Indicador (I)
- Object Inherit
- Container Inherit
- Propagación de permisos entre directorios

Figura 14 — Verificación del permiso heredado de ProjectTeam en ProjectA

```
PS C:\WINDOWS\system32> icacls "C:\CompanyLab\Projects\ProjectA"
C:\CompanyLab\Projects\ProjectA WIN-SFMECCHAUKEK\ProjectTeam:(I)(OI)(CI)(M)
                                WIN-SFMECCHAUKEK\LabAdmins:(I)(OI)(CI)(F)
                                BUILTIN\Administrators:(I)(OI)(CI)(F)
                                NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
                                BUILTIN\Users:(I)(OI)(CI)(RX)
                                NT AUTHORITY\Authenticated Users:(I)(M)
                                NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)
```

14. Verificación Final de Permisos

Como última etapa se verificaron conjuntamente los permisos configurados sobre los principales recursos del laboratorio.

Comandos utilizados

```
icacls "C:\CompanyLab\Public"
icacls "C:\CompanyLab\Projects"
icacls "C:\CompanyLab"
```

La configuración principal obtenida fue:

- LabUsers → Public → Read & Execute (RX)
- ProjectTeam → Projects → Modify (M)
- LabAdmins → CompanyLab → Full Control (F)

Esta comprobación permitió confirmar que los tres niveles de acceso configurados durante el laboratorio permanecían correctamente asignados.

Figura 15 — Verificación final de los permisos configurados

```
PS C:\WINDOWS\system32> icacls "C:\CompanyLab\Public"
C:\CompanyLab\Public WIN-SFMECCHAUKE\LabUsers:(OI)(CI)(RX)
                     WIN-SFMECCHAUKE\LabAdmins:(I)(OI)(CI)(F)
                     BUILTIN\Administrators:(I)(OI)(CI)(F)
                     NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
                     BUILTIN\Users:(I)(OI)(CI)(RX)
                     NT AUTHORITY\Authenticated Users:(I)(M)
                     NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)

Successfully processed 1 files; Failed processing 0 files
PS C:\WINDOWS\system32> icacls "C:\CompanyLab\Projects"
C:\CompanyLab\Projects WIN-SFMECCHAUKE\ProjectTeam:(OI)(CI)(M)
                      WIN-SFMECCHAUKE\LabAdmins:(I)(OI)(CI)(F)
                      BUILTIN\Administrators:(I)(OI)(CI)(F)
                      NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
                      BUILTIN\Users:(I)(OI)(CI)(RX)
                      NT AUTHORITY\Authenticated Users:(I)(M)
                      NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)

Successfully processed 1 files; Failed processing 0 files
PS C:\WINDOWS\system32> icacls "C:\CompanyLab"
C:\CompanyLab WIN-SFMECCHAUKE\LabAdmins:(OI)(CI)(F)
              BUILTIN\Administrators:(I)(OI)(CI)(F)
              NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
              BUILTIN\Users:(I)(OI)(CI)(RX)
              NT AUTHORITY\Authenticated Users:(I)(M)
              NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)
```

15. Pruebas Realizadas

Durante el laboratorio se validó correctamente:

- Creación de usuarios locales.
- Verificación del estado de las cuentas.
- Creación de grupos locales.
- Incorporación de usuarios a grupos.
- Verificación individual y general de membresías.
- Creación de una estructura jerárquica de recursos.
- Inspección de las ACL existentes antes de realizar modificaciones.
- Identificación de permisos predeterminados de Windows.
- Asignación de Read & Execute a LabUsers.
- Asignación de Modify a ProjectTeam.
- Asignación de Full Control a LabAdmins.
- Configuración de herencia mediante OI y CI.
- Identificación de permisos heredados mediante (I).
- Verificación final de los permisos mediante icacls.

16. Análisis

El laboratorio permitió comprender cómo Windows utiliza usuarios, grupos y Access Control Lists para administrar el acceso a los recursos del sistema de archivos.

Uno de los conceptos principales fue la separación entre identidad y autorización. Una cuenta puede encontrarse habilitada sin poseer privilegios administrativos, mientras que el acceso a determinados recursos depende de las membresías de grupo y de las ACL configuradas.

El uso de grupos permitió implementar un modelo de administración organizado. En lugar de asignar directamente todos los permisos a usuarios individuales, se utilizaron **LabUsers**, **ProjectTeam** y **LabAdmins** como grupos a los cuales se otorgaron diferentes niveles de acceso.

También se observaron las diferencias entre **Read & Execute**, **Modify** y **Full Control**, permitiendo relacionar distintos niveles de permisos con diferentes funciones dentro del escenario.

La inspección de la ACL antes de realizar modificaciones permitió comprobar que Windows ya mantiene entradas de acceso para identidades como **Administrators**, **SYSTEM**, **Users** y **Authenticated Users**. Esto permitió diferenciar los permisos existentes del sistema de aquellos agregados específicamente durante el laboratorio.

Finalmente, la comprobación realizada sobre **ProjectA** permitió observar de forma práctica el funcionamiento de la herencia. El permiso **Modify** otorgado a **ProjectTeam** sobre **Projects** se propagó hacia el subdirectorío, siendo identificado mediante el indicador (I).

17. Conclusión

Este laboratorio permitió aplicar conceptos fundamentales de administración de usuarios, grupos y permisos en Windows mediante PowerShell.

Se construyó un escenario práctico en el que diferentes cuentas fueron organizadas mediante grupos y posteriormente se asignaron distintos niveles de acceso sobre recursos del sistema de archivos NTFS.

El uso de **Get-Acl** permitió inspeccionar las listas de control de acceso existentes, mientras que **icacls** permitió configurar y verificar permisos como **Read & Execute**, **Modify** y **Full Control**.

También se comprobó de manera práctica el funcionamiento de la herencia de permisos entre directorios, permitiendo comprender cómo una configuración aplicada sobre un recurso padre puede propagarse hacia sus elementos descendientes.

Estos conceptos constituyen una base importante para tareas de soporte técnico y administración de sistemas Windows, donde la correcta gestión de usuarios, grupos y permisos es fundamental para controlar el acceso a los recursos.

15. Tests Performed

The following tasks were successfully validated during the lab:

- Creation of local users.
- Verification of account status.
- Creation of local groups.
- Addition of users to groups.
- Individual and general verification of group memberships.
- Creation of a hierarchical resource structure.
- Inspection of existing ACLs before making changes.
- Identification of default Windows permissions.
- Assignment of Read & Execute to **LabUsers**.
- Assignment of Modify to **ProjectTeam**.
- Assignment of Full Control to **LabAdmins**.
- Configuration of inheritance using **OI** and **CI**.
- Identification of inherited permissions using **(I)**.
- Final verification of permissions using **icacls**.

16. Analysis

The lab provided an understanding of how Windows uses users, groups, and Access Control Lists to manage access to file system resources.

One of the main concepts was the separation between identity and authorization. An account can be enabled without having administrative privileges, while access to specific resources depends on group memberships and the configured ACLs.

Using groups made it possible to implement an organized administration model. Instead of assigning all permissions directly to individual users, **LabUsers**, **ProjectTeam**, and **LabAdmins** were used as groups to which different levels of access were granted.

The differences between Read & Execute, Modify, and Full Control were also observed, making it possible to associate different permission levels with different functions within the scenario.

Inspecting the ACL before making changes showed that Windows already maintains access entries for identities such as **Administrators**, **SYSTEM**, **Users**, and **Authenticated Users**. This made it possible to distinguish existing system permissions from those specifically added during the lab.

Finally, the verification performed on **ProjectA** demonstrated how inheritance works in practice. The **Modify** permission granted to **ProjectTeam** on **Projects** propagated to the subdirectory and was identified by the **(I)** indicator.

17. Conclusion

This lab provided practical experience with fundamental concepts of user, group, and permission management in Windows using PowerShell.

A practical scenario was built in which different accounts were organized into groups, and different levels of access were subsequently assigned to NTFS file system resources.

`Get-Acl` was used to inspect existing access control lists, while `icacls` was used to configure and verify permissions such as Read & Execute, Modify, and Full Control.

The lab also demonstrated how permission inheritance works between directories, providing an understanding of how a configuration applied to a parent resource can propagate to its descendant objects.

These concepts provide an important foundation for technical support and Windows system administration tasks, where proper management of users, groups, and permissions is essential for controlling access to resources.

Esta es la **traducción completa del contenido de tu PDF**, conservando sus 17 secciones y las 15 figuras. No resumí el *Analysis* ni la *Conclusion*, y los comandos quedaron intactos, como corresponde.